

沈进坤面试题

面试结论

总体评价：做过，但仅限做过——17 年工龄未体现出与之匹配的技术深度。

面试实际只覆盖了前三项核心维度（架构能力、技术深度、数据能力），之后已判定无需继续考察工程能力和领导力。各维度具体表现如下：

架构能力（4-5 / 10）：候选人对 Eureka、Nacos、Spring Cloud 等中间件有使用经验，也确实参与过架构迁移项目。但当追问到 CAP 模型在不同注册中心的工程语义差异、双向同步的一致性边界、以及在高并发/大数据量下的架构取舍时，回答停留在“操作层面”——知道怎么配、怎么迁，但讲不清楚为什么这样选、换一种会怎样。认知深度与预期有显著差距。

技术深度（4-5 / 10）：JVM 相关追问（GC 调优、JIT 编译、逃逸分析）暴露出知识碎片化的问题。候选人对 GC 的基本概念有了解，但无法从 GC 日志反推根因、给不出明确的调优决策链路；JIT 和 deoptimization 方向基本无实战经验。综合判断：技术知识更像“项目用到什么学什么”，缺乏系统性的底层原理框架，17 年工龄下这尤为突出。

数据能力（4 / 10）：候选人使用过多种数据库（SQL Server、MySQL、MongoDB、ES），但深入追问时发现——对 MySQL 优化器的行为细节（如 index dive、统计信息采样）、SQL Server 与 MySQL 的优化器差异、以及大数据量下的分库分表工程方案缺乏系统性认知。数据库认知停留在“能写 SQL、能建索引”的层次，与 17 年工龄预期的数据库选型和调优深度不吻合。

工程能力（/）：前三项面完后已判定不符合岗位要求，未继续考察。

领导力（/）：前三项面完后已判定不符合岗位要求，未继续考察。

核心问题归纳：该候选人的职业轨迹覆盖了广泛的技术栈和多个项目角色，但每次深入追问都停留在“我做过这个”而非“我理解这个”。“做过”只是起点，从“做过”中提炼出可迁移的判断原则、能从原理层解释现象、能在 trade-off 中给出有理有据的决策——这些才是核心能力。

结论：不通过。

面试重点关注方向（预备参考资料，本轮未全部启用）

1. **架构决策的 trade-off 思维：**能否在 CAP、性能 vs 一致性、微服务拆分 vs 运维复杂度之间做出有理有据的权衡，而非简单选边。

2. **技术深度的真实边界**：JVM 底层、分布式一致性、数据库优化器行为——这些是区分"会用框架"和"理解原理"的分水岭。
 3. **商业思维**：不只是解决技术问题，更要把技术投入翻译成业务收益，能否说服非技术决策者是关键。
 4. **坦诚度**：经历过大规模系统的人一定有失败案例，重点关注能否坦诚复盘教训而非只讲成功。
-

第一部分：系统架构与分布式（权重 25%）

题 1：注册中心选型 — Eureka → Nacos 迁移

在数字权益平台项目中做过 Eureka 到 Nacos 的迁移，用了 NacosSync 做过渡。请展开讲：

主问题：Eureka AP 模型和 Nacos 同时支持 AP/CP 模型，在实际迁移过程中，是如何处理服务发现的语义差异的？迁移期间，有没有遇到过"注册中心数据不一致导致流量打到已下线节点"的问题，怎么解决的？

追问方向：

- NacosSync 在双向同步时，如果两边同时有注册/注销操作，冲突如何仲裁？
- Eureka 自保护模式下，所有实例都"可用"——当时怎么判断真实的下线节点？
- 如果让你重来一次，会选 Nacos 的 AP 还是 CP 模式，为什么？

评分标准：

- 只答"NacosSync 自动同步"
 - 能说出 CAP 在注册中心的具体体现和双写冲突
 - 能结合生产故障案例说明自保护机制的坑和双向同步的一致性边界
-

题 2：分布式一致性实践

在 108 社区和智慧城市 SaaS 平台中都用了 RabbitMQ/RocketMQ 做异步通讯。

主问题：假设有一个"用户下单后扣减库存并发放积分"的场景，订单服务、库存服务、积分服务都在独立的数据库中。如何保证这三个操作的数据最终一致性？具体用过什么模式，踩过什么坑？

追问方向：

- RocketMQ 的事务消息和 RabbitMQ 的 publisher confirm + 本地消息表，分别会在什么场景选哪个？

- 如果消费方处理成功了，但发送方本地事务回滚了怎么办？
- 对账机制是怎么做的？T+1 离线对账还是准实时补偿？

评分标准：

- 只答"用 MQ 异步"
 - 能对比不同方案的事务保证力度和适用场景
 - 能从"消息发送+本地事务非原子"出发，讲清半消息/事务回查/补偿的完整链路
-

第二部分：JVM 与性能调优（权重 20%）

题 3：GC 策略实战

主问题：假设 108 社区的线上服务出现了偶发性 Full GC，每次约 2-3 秒，导致部分请求超时。请完整描述排查和解决思路——从发现问题到最终修复的全过程。

追问方向：

- 怎么确定是 GC 导致的 RT 抖动，而不是其他原因（如数据库慢查询、网络抖动）？
- 有没有遇到过"调了 GC 参数后吞吐量反而下降"的情况？为什么？
- 实际项目中最常使用的 GC 组合是什么？如果给你一个 8G 堆、对延迟敏感的服务，你会怎么配参数？

评分标准：

- 只答"调大堆内存"或"换 G1"
 - 能从 GC 日志中提取分析线索，给出调优决策依据
 - 能综合评估"调参数 vs 改代码内存泄漏 vs 增加机器"的收益
-

第三部分：数据库与存储（权重 20%）

题 4：千万级用户系统的 SQL 优化

你的用户系统支撑 400+ 游戏接入，服务了大量用户。

主问题：假设你们的用户表（MySQL）已经达到 5000 万行，索引加了但慢查询仍然很多。请从索引设计、SQL 改写、架构层面三个维度，说说你的优化思路。特别注意：你之前用的是 SQL Server + DDD 架构，和 MySQL 在优化器行为上有哪些差异？

追问方向：

- SQL Server 和 MySQL 在查询优化器上的关键差异是什么？（如统计信息更新策略、索引选

择逻辑)

- `SELECT * FROM user WHERE id IN (...)` 一千个 ID , MySQL 优化器会怎么执行? 会不会走索引? 会不会出现 index dive 问题?
- 如果必须分库分表, 你会选什么中间件? sharding key 怎么选?

评分标准:

- 只答"加索引"
 - 能分析慢查询日志、合理评估索引选择性、区分覆盖索引和回表
 - 能从优化器行为差异、分库分表方案取舍、以及"分库分表带来的分布式事务/跨 shard 查询"问题出发
-